

طراحی الگوریتم‌ها – رشد توابع

Introduction to Algorithm

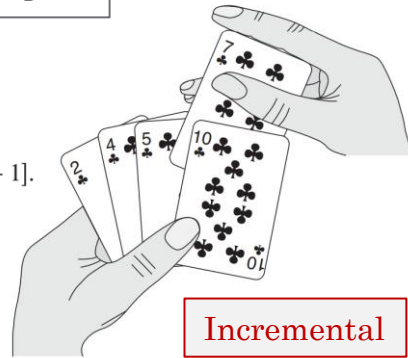
مهدی جوانمردی

مرور مباحث قبل

مرتب‌سازی درجی

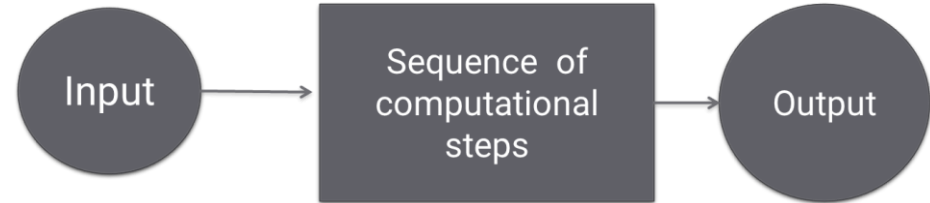
INSERTION-SORT(A)

```
1 for  $j = 2$  to  $A.length$ 
2    $key = A[j]$ 
3   // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4    $i = j - 1$ 
5   while  $i > 0$  and  $A[i] > key$ 
6      $A[i+1] = A[i]$ 
7      $i = i - 1$ 
8    $A[i+1] = key$ 
```



Incremental

الگوریتم



اثبات درستی الگوریتم‌های تکرار

Loop Invariant

```
while (condition) {
  " " " "
  invariant I
  " " " "
  code
}
```

- Initialization
- Maintenance
- ★ Termination

اهمیت طراحی الگوریتم

- کامپیوترها سریع هستند ولی نه بینهایت!
- حافظه ارزان است ولی رایگان نیست!

منابع محدود و مشخص

مرتب‌سازی زمانی

INSERTION-SORT(A)

```
1 for  $j = 2$  to  $A.length$ 
2    $key = A[j]$ 
3   // Insert  $A[j]$  into the sorted
   sequence  $A[1..j-1]$ .
4    $i = j - 1$ 
5   while  $i > 0$  and  $A[i] > key$ 
6      $A[i+1] = A[i]$ 
7      $i = i - 1$ 
8    $A[i+1] = key$ 
```

cost	times
c_1	n
c_2	$n-1$
	0
c_4	$n-1$
c_5	$\sum_{j=2}^n t_j$
c_6	$\sum_{j=2}^n (t_j - 1)$
c_7	$\sum_{j=2}^n (t_j - 1)$
c_8	$n-1$

- Best case
- Worst case
- Average

مرتب‌سازی

Ex. Input: sequence 31, 41, 59, 26, 41, 58

Output: sequence 26, 31, 41, 41, 58, 59

مرور مباحث قبل

روش تقسیم و حل

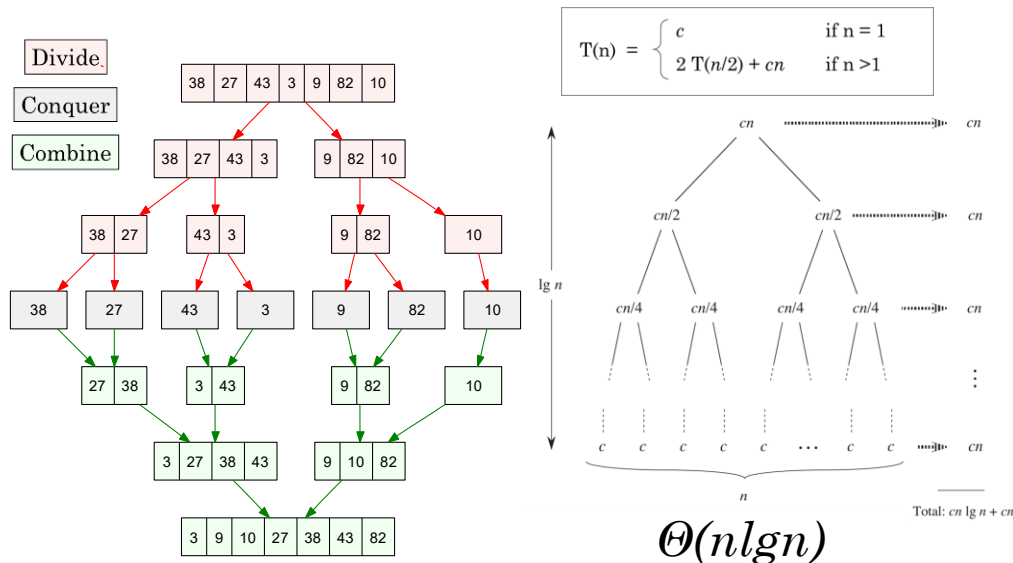
The divide-and-conquer approach:

Divide the problem into a number of subproblems

Conquer the subproblems by solving them recursively

Combine subproblems and solve the original problem

مرتب‌سازی ادغامی

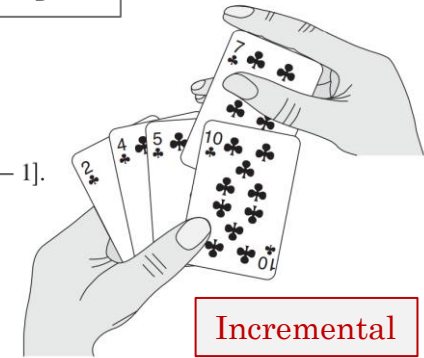


مرتب‌سازی درجی

INSERTION-SORT(A)

```

1 for j = 2 to A.length
2   key = A[j]
3   // Insert A[j] into the sorted sequence A[1..j-1].
4   i = j - 1
5   while i > 0 and A[i] > key
6     A[i + 1] = A[i]
7     i = i - 1
8   A[i + 1] = key
    
```



بدترین و بهترین حالت

Best case The array is already sorted

1	2	3	4	5	6
1	2	3	4	5	6

$\Theta(n)$

Worst case The array is in reverse sorted order

1	2	3	4	5	6
6	5	4	3	2	1

$\Theta(n^2)$

فصل سوم: رشد توابع

- تعریف نمادهای رشد مجانبی
- نمادهای رشد مجانبی در معادلات
- ویژگی‌های مقایسه توابع با نمادهای رشد مجانبی
- توابع مرسوم و نمادهای استاندارد

رشد توابع و نمادهای تقریب مرتبه زمانی

- زمان اجرای دقیق الگوریتم اطلاعات غیر ضرور ← نیاز به زمان اجرای تقریبی یا مرتبه زمانی

- نمادهای تقریب مرتبه زمانی

Name	Notation
Big O(micron)	$\mathcal{O}$ or O
Big Omega	Ω
Big Theta	Θ
Small O(micron)	o
Small Omega	ω

- کاربرد این نمادها در درس طراحی الگوریتم: مقایسه الگوریتمها بر حسب زمان اجرا
- کاربردهای دیگر: تحلیل دیگر ویژگیهای الگوریتم بر حسب تعداد ورودی ← حجم حافظه
- مهم: کدام زمان اجرا؟ بهترین – بدترین – متوسط؟ ← در نماد Θ اهمیت این موضوع بیشتر

رشد توابع و نمادهای تقریب مرتبه زمانی

Name	Notation
Big O(micron)	$\mathcal{O}$ or O
Big Omega	Ω
Big Theta	Θ
Small O(micron)	o
Small Omega	ω

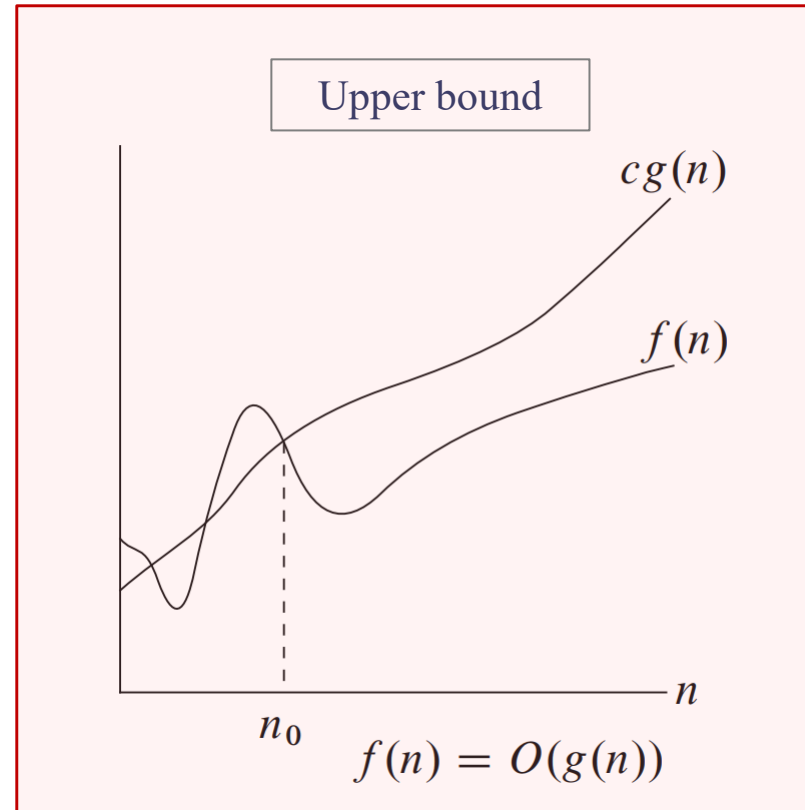
A way to compare “sizes” of functions:

$$\begin{array}{lll}
 O & \approx & \leq \\
 \Omega & \approx & \geq \\
 \Theta & \approx & = \\
 o & \approx & < \\
 \omega & \approx & >
 \end{array}$$

The O -Notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$

$O(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \leq c \cdot g(n)\}$



n_0 : minimum possible

$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)}$ is finite

$n = O(n^2)$

The O -Notation

$$O(g(n)) = \{ f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \leq c \cdot g(n) \}$$

• نماد O : تخمین زمان اجرا از روی ساختار برنامه

$g(n)$ is an *asymptotic upper bound* for $f(n)$.

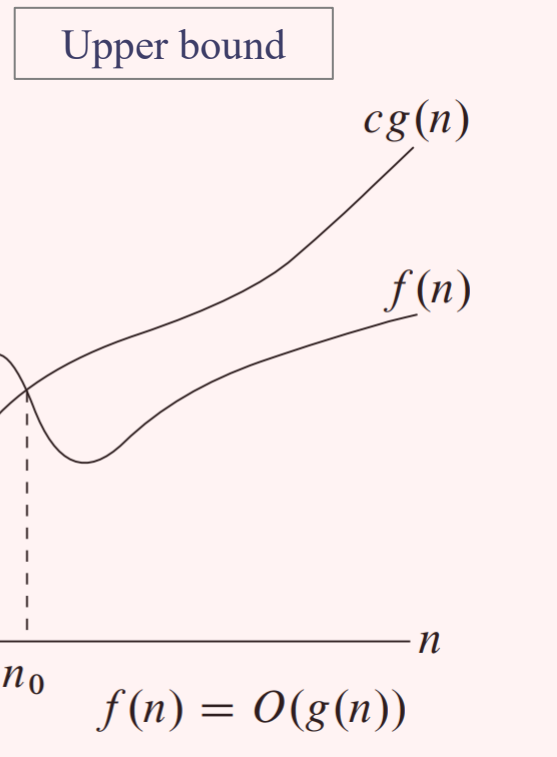
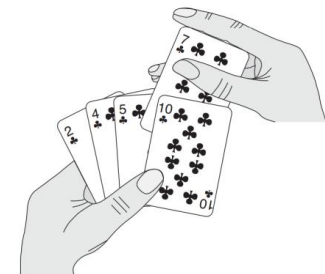
If $f(n) \in O(g(n))$, we write $f(n) = O(g(n))$

INSERTION-SORT(A)

```

1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i + 1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i + 1] = key$ 
```

$$O(n^2)$$



The O -Notation

$$O(g(n)) = \{ f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \leq c \cdot g(n) \}$$

Example

$$2n^2 = O(n^3), \text{ with } c = 1 \text{ and } n_0 = 2.$$

Examples of functions in $O(n^2)$:

$$n^2$$

$$n^2 + n$$

$$n^2 + 1000n$$

$$1000n^2 + 1000n$$

Also,

$$n$$

$$n/1000$$

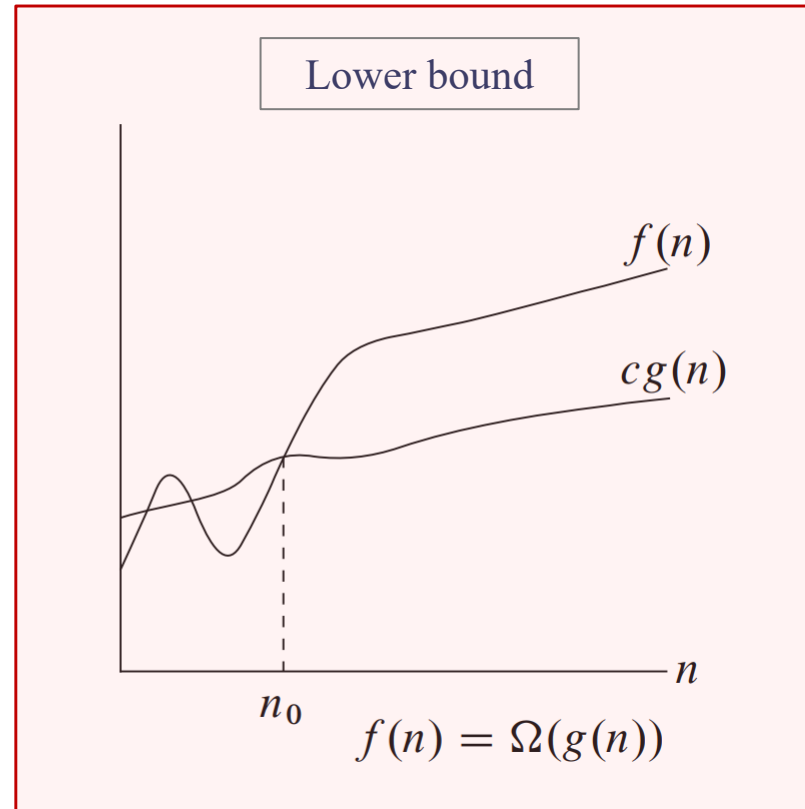
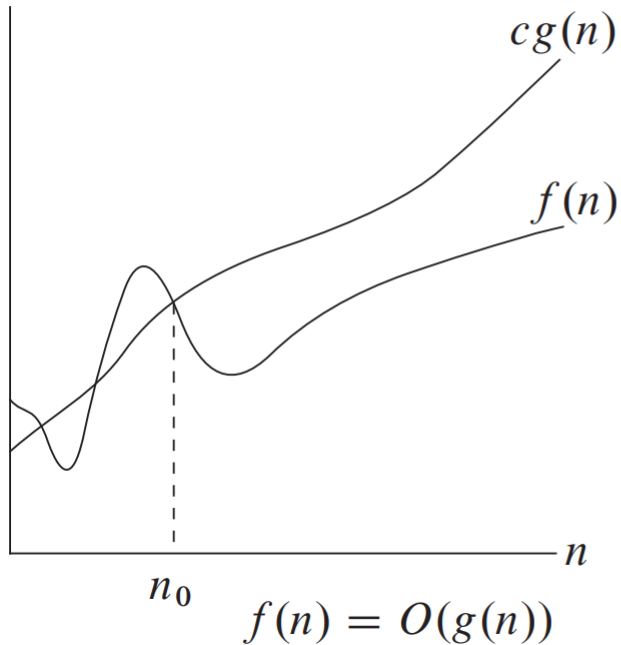
$$n^{1.99999}$$

$$n^2 / \lg \lg \lg n$$

The Ω -Notation

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}.$

$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \geq c \cdot g(n) \geq 0\}$$



n_0 : minimum possible

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} > 0$$

The Ω -Notation

$$\Omega(g(n)) = \{ f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \geq c \cdot g(n) \}$$

• نماد Ω : تخمین زمان اجرا از روی ساختار برنامه
بر اساس بهترین زمان اجرا

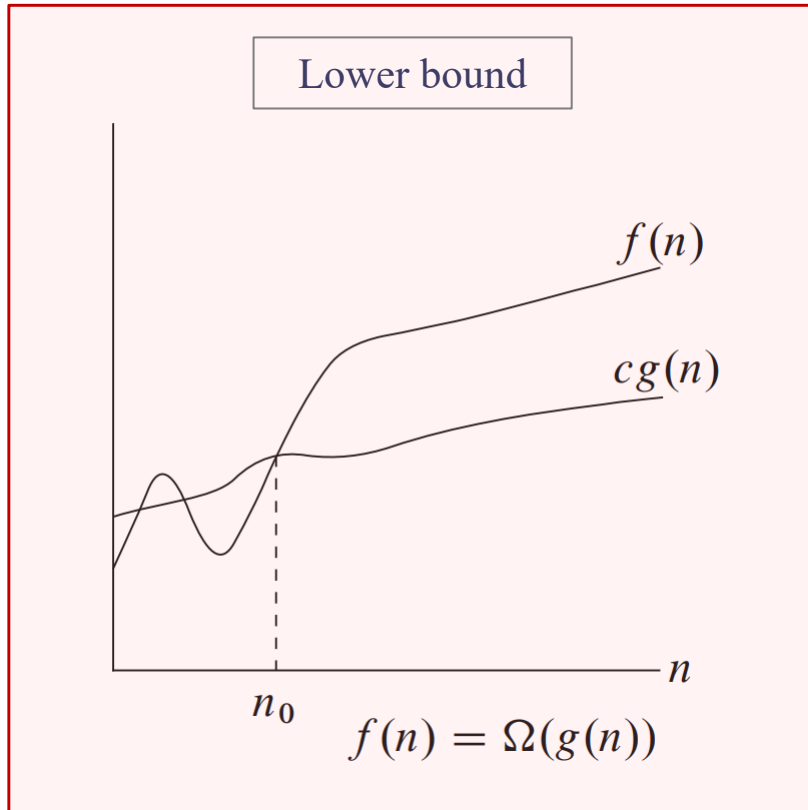
INSERTION-SORT(A)

```

1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i + 1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i + 1] = key$ 
```



$\Omega(n)$



The Ω -Notation

$$\Omega(g(n)) = \{ f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0 : f(n) \geq c \cdot g(n) \}$$

Example

$$\sqrt{n} = \Omega(\lg n), \text{ with } c = 1 \text{ and } n_0 = 16.$$

Examples of functions in $\Omega(n^2)$:

$$n^2$$

$$n^2 + n$$

$$n^2 - n$$

$$1000n^2 + 1000n$$

$$1000n^2 - 1000n$$

Also,

$$n^3$$

$$n^{2.00001}$$

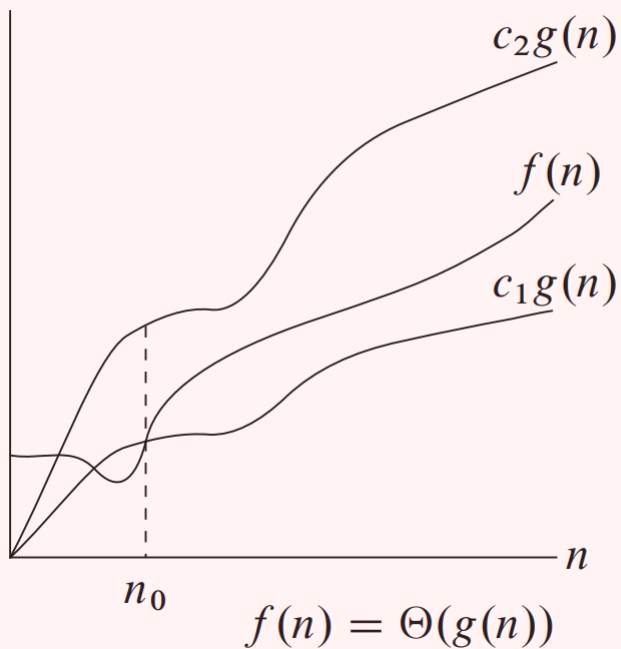
$$n^2 \lg \lg \lg n$$

$$2^{2^n}$$

The Θ -Notation

$$\Theta(g(n)) = \{ f(n) : \exists c_1, c_2 > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Tight bound



A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be “sandwiched” between $c_1g(n)$ and $c_2g(n)$, for sufficiently large n .

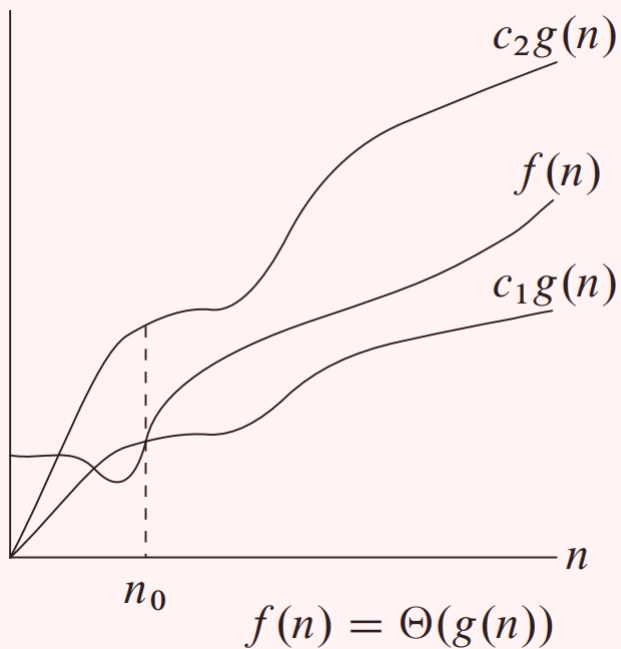
$$\begin{array}{c} \downarrow \\ f(n) \in \Theta(g(n)) \longrightarrow f(n) = \Theta(g(n)) \end{array}$$

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = c$$

The Θ -Notation

$$\Theta(g(n)) = \{ f(n) : \exists c_1, c_2 > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Tight bound



$$\frac{1}{2}n^2 - 3n = \Theta(n^2)$$

?

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$$

for all $n \geq n_0$. Dividing by n^2 yields

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2.$$

$$c_1 = 1/14 \quad c_2 = 1/2 \quad n_0 = 7.$$

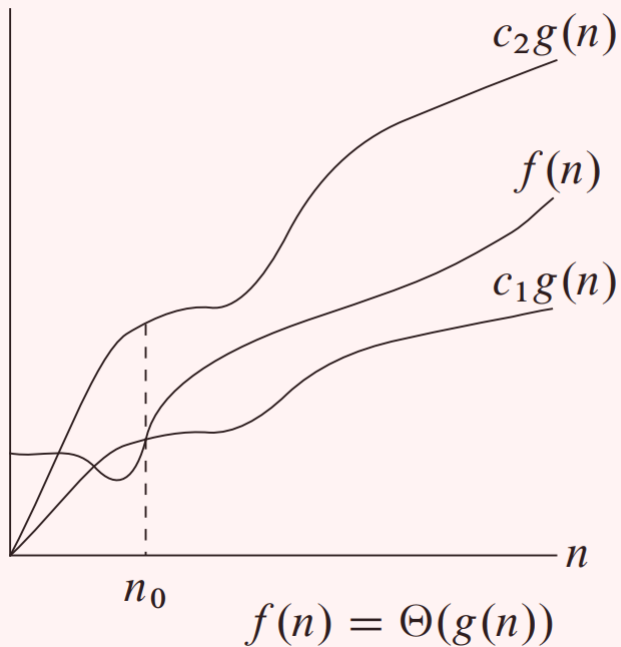
$$6n^3 \neq \Theta(n^2)$$

?

The Θ -Notation

$$\Theta(g(n)) = \{ f(n) : \exists c_1, c_2 > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Tight bound



$$\frac{1}{2}n^2 - 3n = \Theta(n^2)$$

$$6n^3 \neq \Theta(n^2)$$

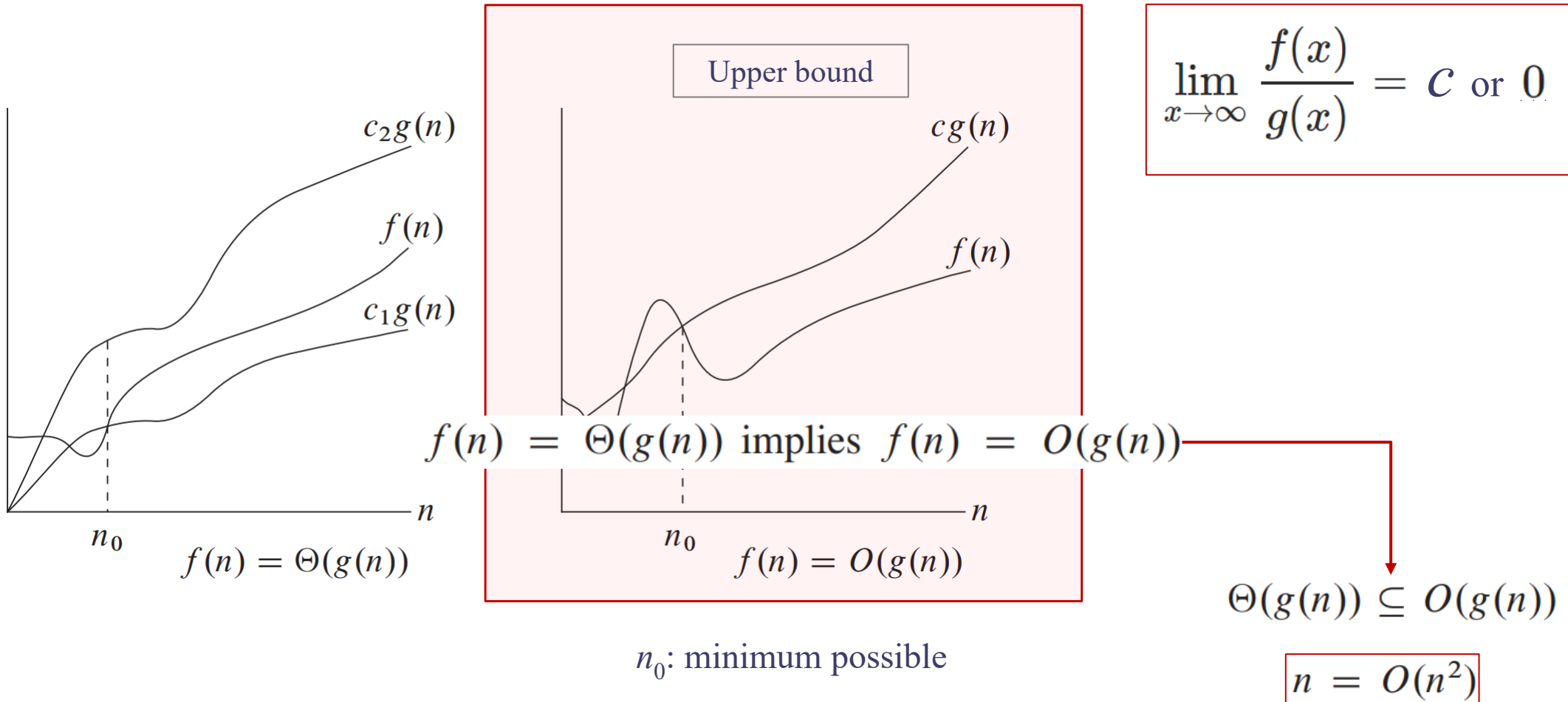
?

We can also use the formal definition to verify that $6n^3 \neq \Theta(n^2)$. Suppose for the purpose of contradiction that c_2 and n_0 exist such that $6n^3 \leq c_2n^2$ for all $n \geq n_0$. But then dividing by n^2 yields $n \leq c_2/6$, which cannot possibly hold for arbitrarily large n , since c_2 is constant.

$$p(n) = \sum_{i=0}^d a_i n^i \quad \text{where the } a_i \text{ are constants and } a_d > 0 \quad p(n) = \Theta(n^d)$$

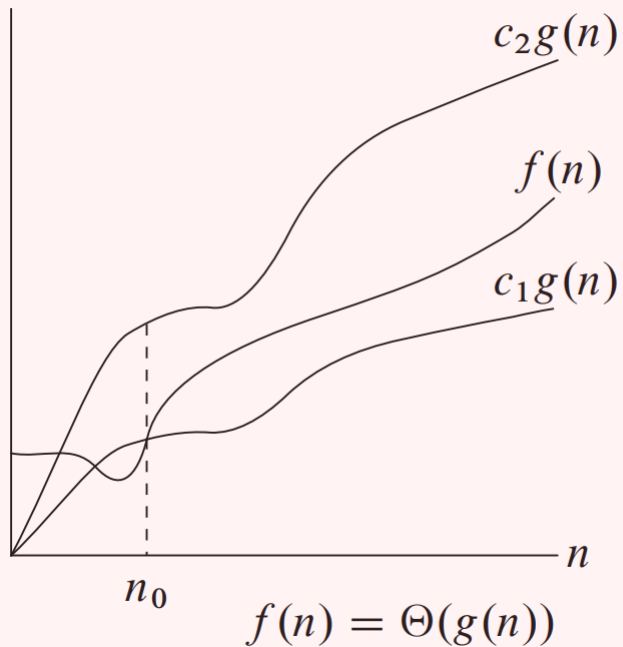
The O -Notation

$$O(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \leq c \cdot g(n)\}$$



The Θ , O and Ω -Notation

Tight bound



Theorem

$f(n) = \Theta(g(n))$ if and only if $f = O(g(n))$ and $f = \Omega(g(n))$

نمادهای رشد مجانبی در معادلات

- حضور نماد رشد مجانبی در سمت راست معادلات

$$2n^2 + 3n + 1 = 2n^2 + f(n)$$

وجود دارد $f(n) \in \Theta(n)$ که معادله صادق باشد

$$f(n) = 3n + 1$$

$$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$$

- حضور نماد رشد مجانبی در سمت چپ معادلات

برای همه توابع $f(n) \in \Theta(n)$

وجود دارد تابع $g(n) \in \Theta(n^2)$

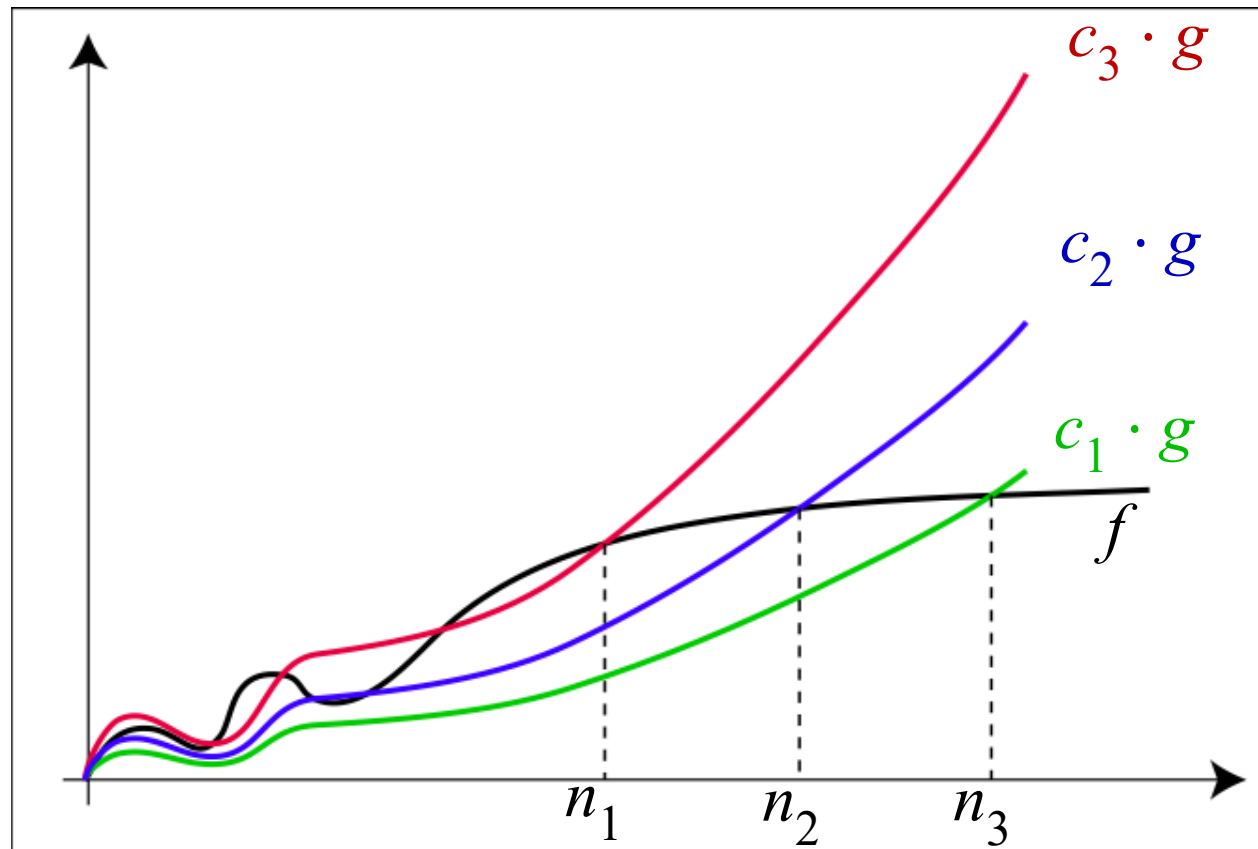
بطوریکه $2n^2 + f(n) = g(n)$

$$2n^2 + \Theta(n) = \Theta(n^2)$$

The o -Notation

$$O(g(n)) = \{ f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0 : f(n) \leq c \cdot g(n) \}$$

$$o(g(n)) = \{ f(n) : \forall c > 0 \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0 : f(n) < c \cdot g(n) \}$$



$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0$$

$$n^{1.9999} = o(n^2)$$

$$n^2 / \lg n = o(n^2)$$

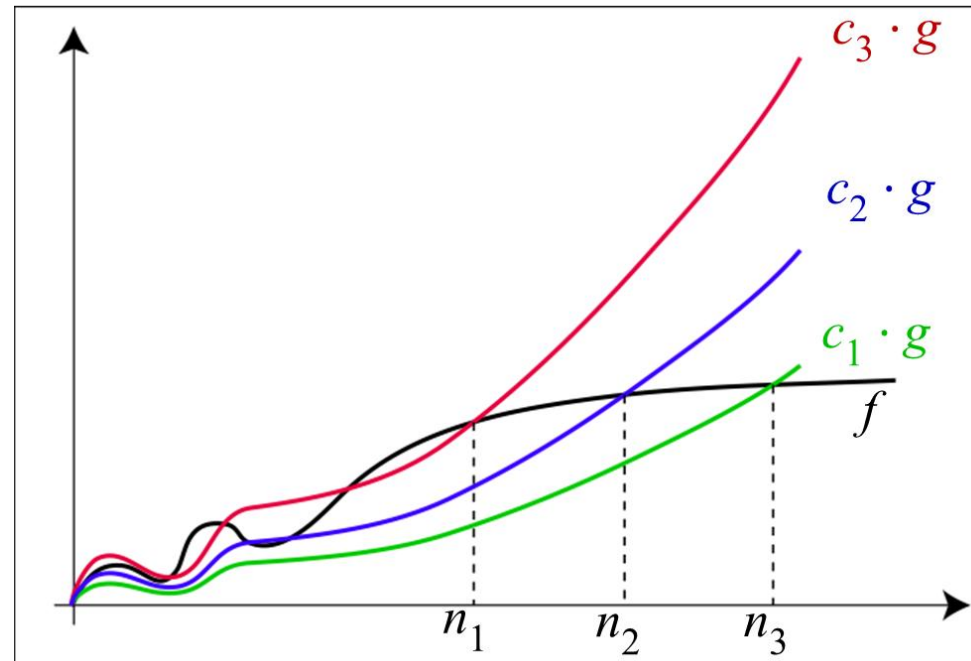
$$n^2 \neq o(n^2) \text{ (just like } 2 \not\leq 2)$$

$$n^2 / 1000 \neq o(n^2)$$

The o -Notation

$$o(g(n)) = \{ f(n) : \forall c > 0 \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0 : f(n) < c \cdot g(n) \}$$

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0$$

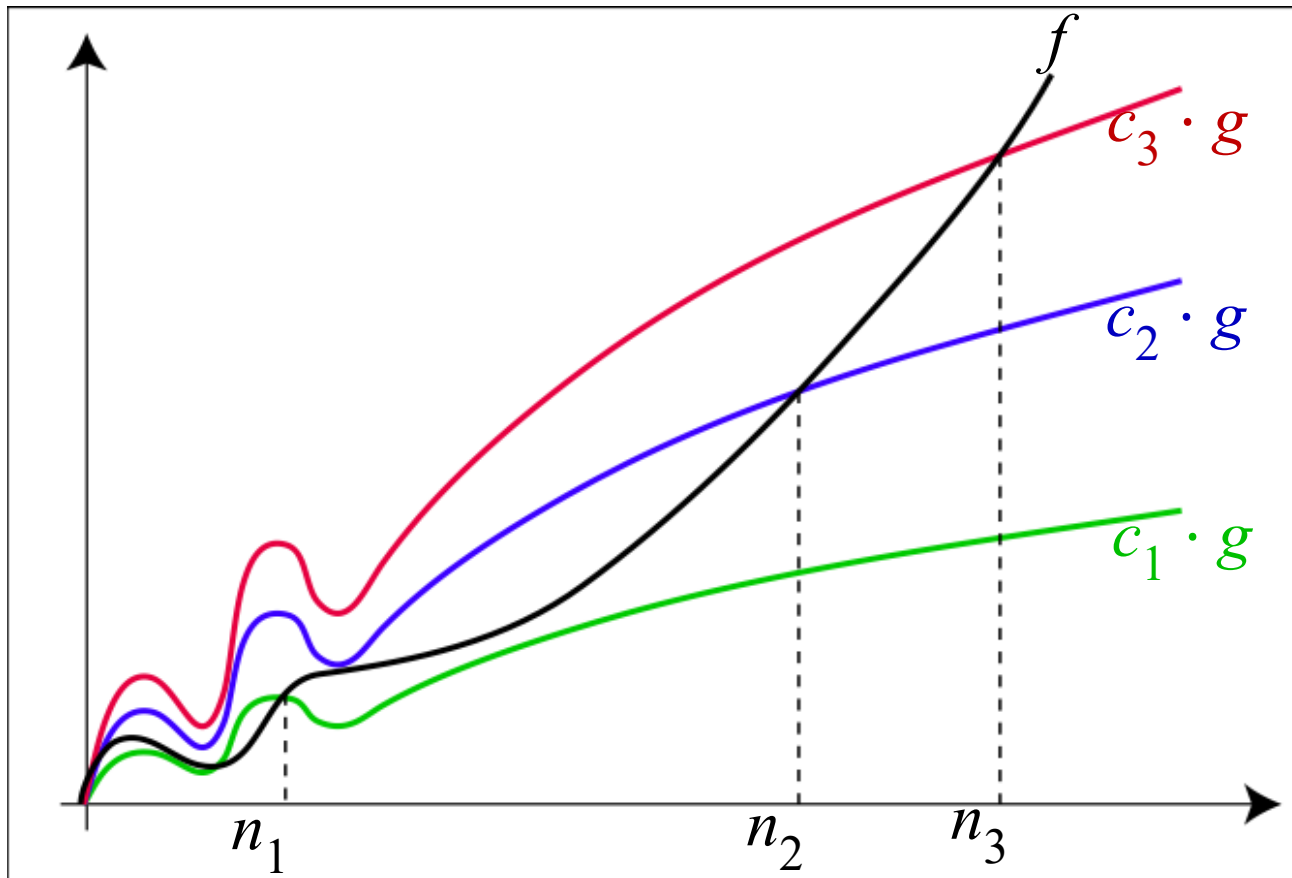


الگوریتم A بهتر است الگوریتم B است ← $T_A(n) = o(T_B(n))$

The ω -Notation

$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) \geq c \cdot g(n)\}$$

$$\omega(g(n)) = \{f(n) : \forall c > 0 \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0: f(n) > c \cdot g(n)\}$$



$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \infty$$

$$\begin{aligned} n^{2.0001} &= \omega(n^2) \\ n^2 \lg n &= \omega(n^2) \\ n^2 &\neq \omega(n^2) \end{aligned}$$

معادل سازی نمادهای رشد مجانبی

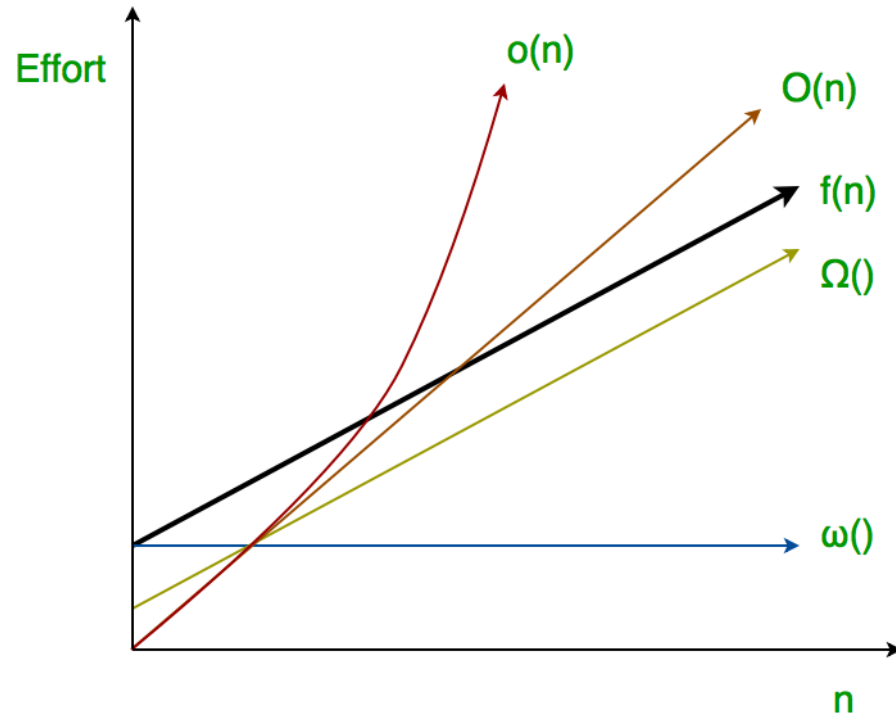
$$f(n) = O(g(n)) \approx a \leq b,$$

$$f(n) = \Omega(g(n)) \approx a \geq b,$$

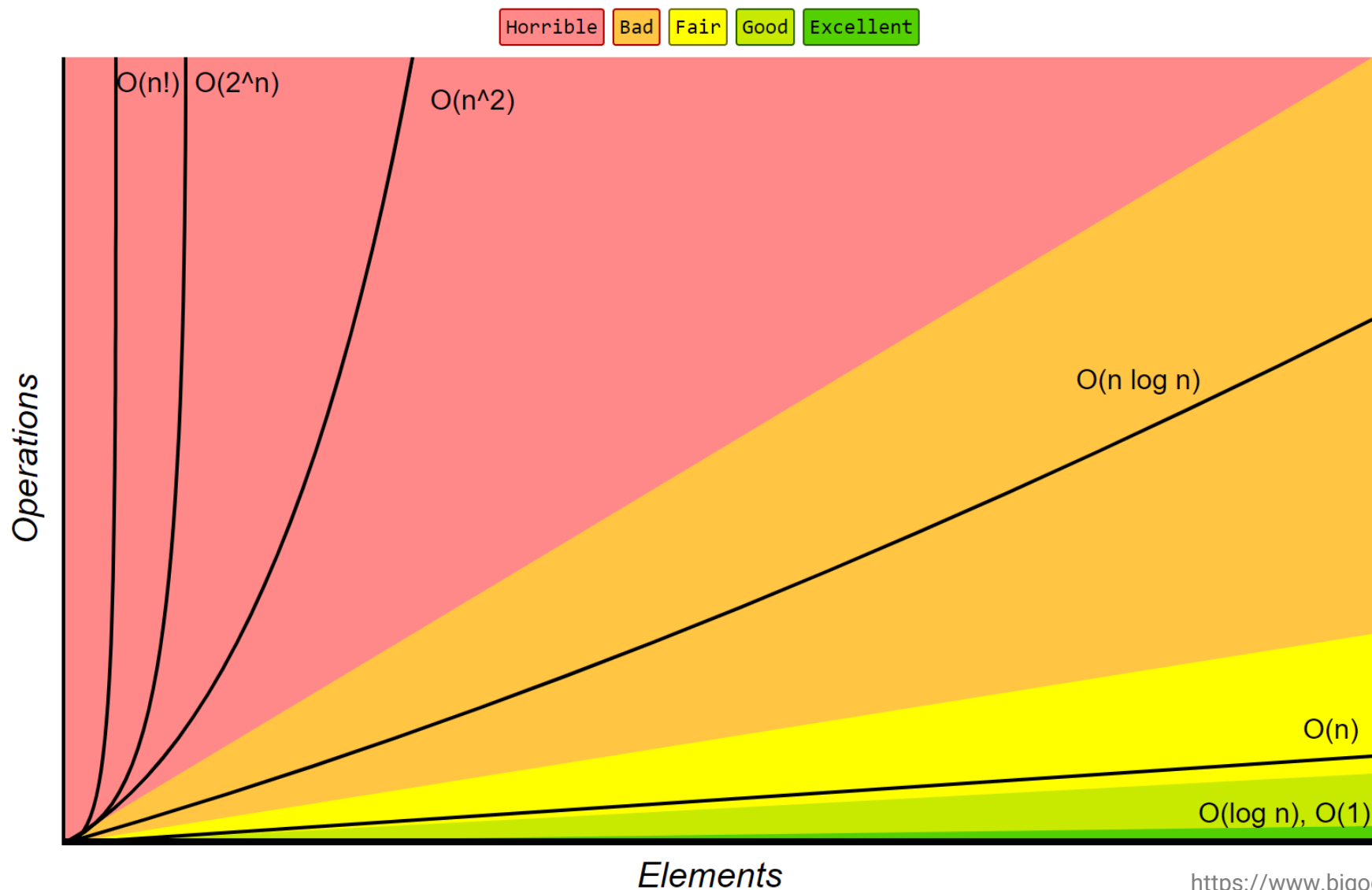
$$f(n) = \Theta(g(n)) \approx a = b,$$

$$f(n) = o(g(n)) \approx a < b,$$

$$f(n) = \omega(g(n)) \approx a > b.$$



مرسوم‌ترین بدترین زمان‌های اجرا



مقایسه خواص توابع

- $f(n) = O(g(n))$ and $g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$
- $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$
- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$

تراگذری

Transitivity

- $f(n) = O(f(n))$ $f(n) \neq o(f(n))$
- $f(n) = \Omega(f(n))$ $f(n) \neq \omega(f(n))$
- $f(n) = \Theta(f(n))$

بازتابی

Reflexivity

مقایسه خواص توابع

- $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$
- $f(n) = O(g(n)) \not\Leftrightarrow g(n) = O(f(n))$
- $f(n) = \Omega(g(n)) \not\Leftrightarrow g(n) = \Omega(f(n))$

تقارنی

Symmetry

- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
- $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

ضد تقارنی

*Transpose
Symmetry*